

理論コードを研究資産として育てるための —Haskell モジュール・import・プロジェクト構成ノート—

Hiroki Funashima

Kindai University Technical College

Sunday, April 26, 2026

1 この文書の狙い

この文書は、個人的なメモではあるが、Haskell コードを単発の教材・試作として終わらせず、今後の研究資産として再利用可能な形へ育てるための考え方を整理したものである。

特に、Haskell 初学者にとって分かりにくく感じやすい

- module
- import
- Main
- src/app

構成を、Python や Java との対応も意識しながら説明する。

本ノートの主眼は二つある。

1. Haskell において、重複実装を避けながらコードをどのように再利用するかを理解すること。
2. その再利用を単なる実務テクニックにとどめず、研究上の共通基盤と個別研究の差分を切り分ける設計へ結びつけること。

本ノートでいう研究資産化とは、単にコードを保存することではない。一度書いたプログラムを、次の授業、次の論文、次の実験、次の理論拡張に再利用できる形へ変換することである。そのためには、実行入口としての Main と、理論的概念・解析手続き・表示処理を担うライブラリ層を分離する必要がある。

Haskell のモジュール分割は、単なるファイル整理ではなく、概念境界をコード上に明示する手段としても読める。特に、型定義、解析関数、アルゴリズム、表示処理を分離することは、理論上の対象、推論手続き、計算方法、解釈表示を分けることに対応している。

2 出発点：車輪の再発明を避けたい

プログラムを毎回一から書くことには、確かに練習としての価値がある。しかし、研究や授業準備を継続していくと、毎回同じ型定義やパーサや補助関数を繰り返し書くのは生産的ではない。ソフトウェア工学の観点から言えば、それは車輪の再発明である。

したがって、手元のコード資産は次のように扱うのが自然である。

- 一度安定した概念定義はライブラリとして切り出す。
- そのライブラリを、別の実行プログラムや実験コードから `import` して使う。
- 研究ごとの差分は、共通基盤の上に薄く重ねる。

Python や Java では、これをモジュール化やパッケージ化として行う。Haskell でも基本発想は同じである。違うのは、ディレクトリを直接指定して使うのではなく、

モジュール名を **import** し、物理配置とモジュール名の対応は **Cabal** の設定に基づいて **GHC** が解決する

という点である。

より正確には、Cabal は `hs-source-dirs` によって GHC が探索すべきソースディレクトリを指定する。その上で、`GameTheory.NormalForm.Types` というモジュール名は、その探索ディレクトリ以下の `GameTheory/NormalForm/Types.hs` に対応する。したがって、`import` 文には物理ディレクトリ名そのものではなく、論理的なモジュール名を書く。

3 Haskell におけるモジュールとは何か

Haskell のモジュールは、ざっくり言えば「関数・型・定数のまとまり」である。module X where と書くと、「このファイルは X という名前の部品です」と宣言していることになる。

例えば、ファイル先頭に

```
module GameTheory.NormalForm.Types where
```

と書けば、そのファイルは GameTheory.NormalForm.Types モジュールになる。

ここに

```
newtype RowStrategy = RowStrategy { unRowStrategy :: String }
newtype ColStrategy = ColStrategy { unColStrategy :: String }

data Payoff = Payoff
  { payoffP1 :: Double
  , payoffP2 :: Double
  }
```

のような定義を書けば、これは「標準形ゲームの型定義のまとまり」という意味を持つ。

3.1 module Main where の意味

Main という名前は特別である。module Main where と main :: IO () を持つファイルは、実行可能プログラムの入口として扱われる。

したがって、Main は「ライブラリ本体」ではなく「入口」である。Haskell では、知的資産の中心は Main に置かず、ライブラリモジュールへ逃がし、Main は薄く保つのがよい設計になる。

4 import は何をしているか

import は、他のモジュールをこのファイルの中で使えるようにする宣言である。発想としては Python の import や Java の import とかなり近い。

例えば、

```
import GameTheory.NormalForm.Types
import GameTheory.NormalForm.Parser
```

と書けば、GameTheory.NormalForm.Types と GameTheory.NormalForm.Parser に定義された型や関数を使える。

4.1 全部 import する場合

```
import Data.List
```

これは Data.List が公開しているものをそのまま持ってくる書き方である。

4.2 必要なものだけ import する場合

```
import Data.List (intercalate, nubBy)
```

これは、Data.List のうち intercalate と nubBy だけを使う、という意味である。何を使っているかが明確になるので、読みやすきの点で利点がある。

4.3 qualified を使う場合

```
import qualified Data.Vector as V
```

これは、Data.Vector を V. という接頭辞付きで使う、という意味である。したがって、コード中では

```
V.length
V.generate
V.fromList
```

のように書く。

この書き方の利点は、名前衝突を避けやすいことにある。例えば length はリストにもベクトルにもあるが、V.length と書けばベクトル用だとすぐ分かる。

5 Python との違い：ディレクトリを直接 import しない

Haskell の import で最初に戸惑いやすい点は、ディレクトリ名を直接 **import** しないことである。
例えば、物理的に

```
my-game-theory/  
  src/  
    GameTheory/  
      NormalForm/  
        Types.hs  
        Parser.hs  
  app/  
    Main.hs
```

という構成にしても、app/Main.hs から

```
import src.GameTheory.NormalForm.Types
```

のように書かない。

正しくは、モジュール名を書いて

```
import GameTheory.NormalForm.Types  
import GameTheory.NormalForm.Parser
```

とする。

つまり Haskell では、

- import 文は論理名で書く。
- その論理名と物理配置の対応は、Cabal の設定に基づいて GHC が解決する。

という二段構えになっている。

この点では、Python より Java の package にやや近い感触を持つ。

6 モジュール名とファイル名の対応

通常、

```
module GameTheory.NormalForm.Types where
```

と書いたら、ファイルは

```
GameTheory/NormalForm/Types.hs
```

に置く。

つまり、モジュール名の `.` はディレクトリ区切りに対応している。

例を並べると、次のようになる。

モジュール名	ファイル位置
<code>GameTheory.NormalForm.Types</code>	<code>src/GameTheory/NormalForm/Types.hs</code>
<code>GameTheory.NormalForm.Parser</code>	<code>src/GameTheory/NormalForm/Parser.hs</code>
<code>GameTheory.NormalForm.Equilibrium.Mixed</code>	<code>src/GameTheory/NormalForm/Equilibrium/Mixed.hs</code>

この対応を揃えておくと、プロジェクトの構造がかなり分かりやすくなる。

7 推奨ディレクトリ構成

例えば、ゲーム理論コードを将来の研究資産として育てるなら、次のような構成が分かりやすい。

```
my-game-theory/  
  my-game-theory.cabal  
  README.md  
  CHANGELOG.md  
  LICENSE  
  src/  
    GameTheory/  
      NormalForm/  
        Types.hs  
        Parser.hs  
        BestResponse.hs  
        Pretty.hs  
        Analysis.hs  
        Equilibrium/  
          Pure.hs  
          Mixed.hs  
          SupportEnumeration.hs  
      ExtensiveForm/  
        Types.hs  
        Parser.hs  
        Viewer.hs  
      Common/  
        Rational.hs  
        Matrix.hs  
        Pretty.hs  
  app/  
    Main.hs  
  test/  
    Spec.hs  
  data/  
    examples/  
      prisoners_dilemma.txt  
      matching_pennies.txt  
      battle_of_the_sexes.txt  
      coordination_game.txt  
      degenerate_game.txt  
  bench/
```

ここでの考え方は単純である。

- src/ にはライブラリを置く。
- app/ には実行可能プログラムの入口を置く。
- test/ にはテストを書く。

- `data/examples/` には授業・実験・再現確認用の入力例を置く。
- `bench/` には性能比較やアルゴリズム比較のためのベンチマークを置く。

研究資産として見ると、モジュール分割は次の層に対応する。

層	役割	例
理論語彙層	対象概念を定義する	Types
入力層	外部表現を内部表現へ変換する	Parser
解析層	best response などの基本構造を計算する	BestResponse
均衡計算層	純粋・混合均衡を求める	Equilibrium.Pure, Equilibrium.Mixed
出力層	人間向けに整形する	Pretty
実験層	研究ごとの比較・検証を行う	Experiment, bench/

8 app/Main.hs からどう import するか

この構成にした場合、app/Main.hs からは次のように import する。

```
module Main where

import GameTheory.NormalForm.Types
import GameTheory.NormalForm.Parser
import GameTheory.NormalForm.BestResponse
import GameTheory.NormalForm.Pretty
import GameTheory.NormalForm.Analysis
import GameTheory.NormalForm.Equilibrium.Pure
import GameTheory.NormalForm.Equilibrium.Mixed

main :: IO ()
main = putStrLn "Hello"
```

ここで重要なのは、src/ というディレクトリ名を import に書いていないことである。書くのはあくまでモジュール名だけである。

9 なぜこれで見つかるのか：Cabal の役割

それでは、なぜ `app/Main.hs` から `src/` 配下のモジュールが見つかるのか。その鍵が `.cabal` ファイルである。

Cabal では、`src` をライブラリのソースディレクトリ、`app` を実行ファイルのソースディレクトリとして宣言できる。

典型例は次のようになる。

```
cabal-version:      3.0
name:               my-game-theory
version:            0.1.0.0
build-type:         Simple

library
  hs-source-dirs:    src
  exposed-modules:
    GameTheory.NormalForm.Types
    , GameTheory.NormalForm.Parser
    , GameTheory.NormalForm.BestResponse
    , GameTheory.NormalForm.Pretty
    , GameTheory.NormalForm.Analysis
    , GameTheory.NormalForm.Equilibrium.Pure
    , GameTheory.NormalForm.Equilibrium.Mixed
  other-modules:
    GameTheory.NormalForm.Equilibrium.SupportEnumeration
    , GameTheory.Common.Rational
    , GameTheory.Common.Matrix
    , GameTheory.Common.Pretty
  build-depends:
    base >=4.14 && <5
    , vector
    , text
    , megaparsec
  default-language: Haskell2010
  ghc-options: -Wall

executable my-game-theory
  hs-source-dirs:    app
  main-is:           Main.hs
  build-depends:
    base >=4.14 && <5
    , my-game-theory
  default-language: Haskell2010
  ghc-options: -Wall
```

ここで重要なのは executable 側の

```
build-depends: my-game-theory
```

である。これは「app/Main.hs は、このパッケージの library を使います」という宣言になっている。したがって、Main.hs ではモジュール名を書くだけでライブラリ側を import できる。

9.1 exposed-modules と other-modules

Cabal の exposed-modules は、ライブラリの外部から import 可能にする公開モジュールを列挙する項目である。一方で、ライブラリ内部では使うが、外部には直接触らせたくない補助モジュールは other-modules に置くことができる。

研究資産としてコードを育てる場合、この区別は重要である。すべての実装詳細を公開してしまうと、後から内部構造を変更しにくくなる。したがって、安定した概念層や公開 API は exposed-modules に置き、アルゴリズム内部の補助関数や実験的実装は other-modules に置く、という方針が自然である。

例えば、外部利用者に直接触らせたい型定義や解析入口は exposed-modules に置く。一方、行列計算や support enumeration の内部補助関数は Internal 以下に置き、other-modules として扱うことができる。

```
library
  hs-source-dirs: src
  exposed-modules:
    GameTheory.NormalForm.Types
    , GameTheory.NormalForm.Parser
    , GameTheory.NormalForm.Analysis
  other-modules:
    GameTheory.NormalForm.Internal.Matrix
    , GameTheory.NormalForm.Internal.Support
```

このように公開 API と内部実装を分けておくと、外部から見える理論的語彙を保ったまま、内部アルゴリズムを後から変更しやすくなる。

10 モジュール分割の意味：何をどこへ置くか

現在のゲーム理論ソルバを一枚の `Main.hs` から分解するとしたら、次の切り分けが自然である。

10.1 `GameTheory.NormalForm.Types`

ここには標準形ゲームの概念定義を置く。

- `RowStrategy`
- `ColStrategy`
- `Payoff`
- `Cell`
- `Game`
- `MarkedCell`
- `PureNashEquilibrium`
- `MixedNashEquilibrium`

これは研究的にも重要である。なぜなら、ここが「この理論で何を語彙として採用するか」を定める層だからである。

10.2 `GameTheory.NormalForm.Parser`

ここには入力テキストを `Game` に変換する処理を置く。

- `Parser`
- `parseStrategyToken`
- `payoffParser`
- `gameParser`
- `parseGameText`
- `parseGameFile`

10.3 `GameTheory.NormalForm.BestResponse`

ここには基本解析を置く。

- 安全アクセス
- `bestResponsesP1Indices`
- `bestResponsesP2Indices`
- `markedMatrix`

10.4 GameTheory.NormalForm.Equilibrium.Pure

ここには純粋戦略均衡の抽出を置く。

- pureNashEquilibria

10.5 GameTheory.NormalForm.Equilibrium.Mixed

ここには混合戦略均衡の探索を置く。

- 組合せ列挙
- ガウス・ジョルダン法
- support enumeration
- 候補検証
- 重複除去

これは今後もっとも差し替えやすい研究層でもある。例えば、より一般の support enumeration、厳密計算版、退化対応版などを比較しやすくなる。

10.6 GameTheory.NormalForm.Pretty

ここには表示関連を置く。計算と表示を分離することで、ロジックの再利用性が上がる。

10.7 GameTheory.NormalForm.Analysis

ここには全体レポートや解析フローを置く。あるいは、さらに進めて表示非依存の解析レポート型を作ってもよい。

11 なぜこの分割が研究に効くのか

ここで重要なのは、モジュール分割が単なる整理整頓ではないことである。研究上は、**共通基盤**と**個別研究の差分を分離**するという意味を持つ。

例えば、

- 型定義
- パーサ
- best response
- 基本表示

は共通基盤として比較的安定している。

一方で、

- 混合戦略均衡の探索アルゴリズム
- 厳密計算か近似計算か
- 展開形ゲームへの拡張
- 退化ゲームへの対応
- ネットワークゲームや TDA との接続

は個別研究ごとの差分になりやすい。

この二層を分けておくと、「毎回全部書き直す」のではなく、「共通基盤の上に今回の研究の新規部分を載せる」という構造になる。これは生産性の面でも、論文執筆の面でも非常に有効である。

12 公開 API としての export list

Haskell のモジュールでは、単に `module X where` と書くだけでなく、外部に公開する名前を明示することもできる。

```
module GameTheory.NormalForm.Types
  ( RowStrategy(..)
  , ColStrategy(..)
  , Payoff(..)
  , Cell(..)
  , Game(..)
  , PureNashEquilibrium(..)
  , MixedNashEquilibrium(..)
  ) where
```

この書き方をすると、モジュール外部から利用可能な型や関数を明示できる。研究資産としてコードを育てる場合、これは単なる情報隠蔽ではなく、理論上の語彙をどこまで公開するかを定める設計判断でもある。

逆に、内部表現や補助関数を `export list` から外しておけば、外部利用者に依存させたくない実装詳細を隠せる。これは、研究コードを後から拡張・修正するときに重要である。

13 Haskell では抽象化しすぎない方がよい場合がある

Haskell を学び始めると、すぐに型クラス化や高度な抽象化をしたくなることがある。しかし、研究基盤を作る初期段階では、抽象化のやりすぎは必ずしも得策ではない。

例えば最初から、

- 任意の利得型
- 任意のゲーム表現
- 任意の配列実装

のように一般化しすぎると、研究資産というより抽象化実験になりやすい。

むしろ最初は、`GameTheory.NormalForm.Types` のように対象をかなり明示したモジュール名にして、理論構造に沿って切る方がよい。この方針は、概念構造を明示しながら研究コードを育てるという目的とも相性がよい。

14 最初の一步として何をやるべきか

いきなり完全な再設計をする必要はない。現実的には、次の順で十分である。

1. `GameTheory.NormalForm.Types` を切り出す。
2. `GameTheory.NormalForm.Parser` と `GameTheory.NormalForm.Pretty` を切り出す。
3. `GameTheory.NormalForm.BestResponse` と `GameTheory.NormalForm.Equilibrium.Pure` を切り出す。
4. `GameTheory.NormalForm.Equilibrium.Mixed` を独立させる。
5. `Main.hs` を薄くして、入口だけにする。

この順序にすると、最初に概念の中心を安定化し、次に周辺部を整理し、最後にアルゴリズムの差分を独立させることができる。

15 理解のための最小例

最後に、最小の雛形を示す。

15.1 src/GameTheory/NormalForm/Types.hs

```
module GameTheory.NormalForm.Types
  ( RowStrategy(..)
  , ColStrategy(..)
  , Game(..)
  ) where

newtype RowStrategy = RowStrategy { unRowStrategy :: String }
  deriving (Eq, Ord, Show)

newtype ColStrategy = ColStrategy { unColStrategy :: String }
  deriving (Eq, Ord, Show)

data Game = Game
  deriving (Show)
```

15.2 src/GameTheory/NormalForm/Parser.hs

```
module GameTheory.NormalForm.Parser
  ( parseGameText
  ) where

import GameTheory.NormalForm.Types

parseGameText :: String -> Either String Game
parseGameText _ = Right Game
```

15.3 app/Main.hs

```
module Main where

import GameTheory.NormalForm.Types
import GameTheory.NormalForm.Parser

main :: IO ()
main = do
  let input = "& L & R\nU & (3,2) & (1,1)\nD & (0,0) & (2,3)\n"
  case parseGameText input of
    Left err -> putStrLn err
```

```
Right g  -> print g
```

この最小例が通れば、Haskell における「ライブラリを切り出し、Main から import して使う」という基本形はつかめていると考えてよい。

16 研究資産としてのテスト

研究用コードでは、単に動くことよりも、理論的性質が保たれていることが重要である。したがって、`test/` には単体テストだけでなく、ゲーム理論上の性質を確認するテストを書くのが望ましい。

例えば、純粋戦略ナッシュ均衡については、抽出されたセルが各プレイヤーにとって同時に best response であることを検査できる。また、混合戦略均衡については、確率の総和が 1 であること、支持集合上の期待利得が等しいこと、支持集合外の戦略がそれを上回らないことを検査できる。

より具体的には、次のようなテストが考えられる。

- 利得行列の行数・列数が戦略数と一致していること。
- pure Nash equilibrium と best response marking が整合していること。
- 混合均衡候補の確率和が 1 になること。
- 支持集合上の期待利得が各プレイヤーで等しくなること。
- 支持集合外の戦略が均衡利得を上回らないこと。
- パーサで読んだゲームを pretty print して、再解釈できること。

このようなテストは、単なるプログラムのバグ検出だけでなく、理論的性質をコード上で維持するための仕組みになる。

17 入力例を保存する意味

ゲーム理論コードでは、`data/examples/` に代表的なゲームを保存しておくといよい。例えば、

- 囚人のジレンマ
- マッチングペニー
- 男女の争い
- 調整ゲーム
- 退化ゲーム

などである。

これは授業用にも研究用にも有効である。授業では典型例をすぐに実行できる。研究では、アルゴリズム変更前後の出力差分を確認できる。また、論文や GitHub 公開時には、再現可能性を高める資料にもなる。

18 まとめ

要点を短くまとめる。

1. Haskell でも、再利用したいコードはモジュールに分けて `import` する。
2. `import` ではディレクトリ名ではなくモジュール名を書く。
3. 物理配置とモジュール名の対応は、Cabal の設定に基づいて GHC が解決する。
4. `Main` は入口であり、研究資産の中心は `src` 側のライブラリに置く。
5. `GameTheory.NormalForm.*` のように対象領域を明示した名前空間にすると、将来の拡張に強くなる。
6. `exposed-modules` と `other-modules` を分けることで、公開 API と内部実装を区別できる。
7. 共通基盤と個別研究の差分を分けることで、生産性と研究の見通しが両立する。

したがって、ゲーム理論の Haskell コードを今後も育てていくなら、一枚の `Main.hs` として持ち続けるより、`GameTheory.NormalForm` 系のライブラリとして切り出していく方がよい。その方が、教育・実装・研究のどの観点から見ても資産化しやすい。

さらに将来的には、`GameTheory.ExtensiveForm`、`GameTheory.Common`、あるいはネットワークゲームや TDA との接続部分を別モジュールとして追加できる。この意味で、モジュール分割は単なるコード整理ではなく、研究テーマの展開可能性を支える設計でもある。

付録 A Haskell 開発環境を構成する主要ツール

本文では、Haskell のモジュール、`import`、`src/app` 構成、Cabal ファイルの役割を説明した。しかし、Haskell の開発環境には複数の道具が登場するため、初学者にとっては GHC、Cabal、Stack、GHCup の関係が分かりにくい。

そこで本 Appendix では、これらの道具がそれぞれ何を担当しているのかを整理する。重要なのは、これらが互いに競合する完全な代替物ではなく、異なる層を担当する道具だという点である。

道具	主な役割	おおまかな位置づけ
GHC	Haskell コンパイラ本体	コードをコンパイル・実行する中核
GHCi	対話環境	小さな式や関数を試す REPL
Cabal	パッケージ記述・ビルド	<code>.cabal</code> に基づく標準的なビルド基盤
Stack	プロジェクトビルド	再現性を重視したビルドツール
GHCup	ツールチェーン管理	GHC, Cabal, HLS, Stack などの導入・切替

付録 B GHC : Haskell を実際にコンパイルする処理系

GHC は Glasgow Haskell Compiler の略であり、現在の Haskell 開発における事実上の標準的な処理系である。Haskell のソースコードを実際にコンパイルするのは、この GHC である。

例えば、単一ファイルの Haskell プログラムなら、

```
ghc Main.hs
```

のようにしてコンパイルできる。

また、対話的に式や関数を試すための環境として GHCi がある。

```
ghci
```

あるいは、ファイルを読み込んで試す場合は、

```
ghci Main.hs
```

のように起動できる。

GHCi は、Haskell の型推論や関数の挙動を確認するために非常に有用である。例えば、授業用コードや研究用コードを書いている途中で、

```
:t someFunction
```

と入力すれば、`someFunction` の型を確認できる。

したがって、GHC はコンパイルの中核であり、GHCi は探索・確認・試行錯誤のための対話的環境である。

付録 C Cabal : Haskell プロジェクトの構成を記述する仕組み

Cabal は、Haskell のライブラリや実行可能プログラムをどのようにビルドするかを記述するための仕組みである。本文で見たように、`.cabal` ファイルには、

- パッケージ名
- バージョン
- ライブラリのソースディレクトリ
- 公開モジュール
- 内部モジュール
- 依存パッケージ
- 実行可能ファイル
- テスト

などを記述する。

例えば、本文の構成では、

```
library
hs-source-dirs:  src
exposed-modules:
    GameTheory.NormalForm.Types
    , GameTheory.NormalForm.Parser
build-depends:
    base >=4.14 && <5
default-language: Haskell2010
```

のように書いた。

これは、src 以下をライブラリのソースディレクトリとし、GameTheory.NormalForm.Types や GameTheory.NormalForm.Parser を公開モジュールとして扱う、という意味である。

実際のビルドは、プロジェクトルートで

```
cabal build
```

を実行する。

実行可能ファイルを動かす場合は、

```
cabal run
```

を使う。

テストを定義している場合は、

```
cabal test
```

でテストを実行できる。

C.1 Cabal ファイルは設計図である

重要なのは、.cabal ファイルが単なる設定ファイルではなく、プロジェクトの設計図だという点である。

特に研究コードでは、

- どのモジュールを公開 API とするか
- どのモジュールを内部実装に閉じるか
- どの依存パッケージに基づいているか
- 実験用 executable とライブラリ本体をどう分けるか

が明示される。

したがって、Cabal ファイルは、研究資産としてのコードの構造を外部から読める形にする役割を持つ。

付録 D Stack：再現性を重視した Haskell プロジェクトビルドツール

Stack は、Haskell プロジェクトをビルドするための別の道具である。Cabal と同じく Haskell プロジェクトをビルドするために使われるが、特に再現性を重視した開発でよく使われる。

Stack プロジェクトでは、通常、

```
stack.yaml
```

というファイルが置かれる。このファイルには、どの GHC バージョンやどのパッケージ集合を使うかが記述される。

典型的な操作は次のようになる。

```
stack build
stack run
stack test
```

それぞれ、

- `stack build` : ビルドする
- `stack run` : 実行する
- `stack test` : テストする

という意味である。

D.1 Cabal と Stack の関係

Cabal と Stack は、初学者には競合する道具のように見えるかもしれない。しかし、より正確には、Stack も内部的には Cabal のパッケージ記述を利用する。つまり、`.cabal` ファイルは Stack プロジェクトでも重要である。

大まかには、次のように理解するとよい。

観点	Cabal 中心	Stack 中心
設定ファイル	<code>.cabal</code> , <code>cabal.project</code>	<code>stack.yaml</code> , <code>.cabal</code>
重視する点	標準的なパッケージ記述・ビルド	再現性のあるビルド環境
典型コマンド	<code>cabal build</code>	<code>stack build</code>
依存管理	Cabal solver	resolver/snapshot ベース

研究コードでは、どちらを使ってもよい。ただし、現在の Haskell 開発では、Cabal 単体でも十分に実用的である。一方、複数人で同一環境を再現したい場合や、特定の resolver に固定したい場合には Stack が便利である。

付録 E GHCup : Haskell ツールチェーンの導入と切替

GHCup は、Haskell 開発に必要なツールチェーンを導入・管理するための道具である。具体的には、

- GHC
- Cabal
- HLS, Haskell Language Server
- Stack

などをインストールしたり、バージョンを切り替えたりするために使う。

例えば、GHC の利用可能なバージョンを確認するには、

```
ghcup list
```

を使う。

特定の GHC をインストールするには、

```
ghcup install ghc 9.8.4
```

のようにする。

使用する GHC を切り替えるには、

```
ghcup set ghc 9.8.4
```

のようにする。

GHCup は、Rust における rustup や、Python における pyenv に近いものとして理解できる。つまり、Haskell のコンパイラや周辺ツールそのものを管理する層である。

付録 F HLS：エディタ連携のための補助ツール

本文の主題からは少し外れるが、実際の開発では HLS、すなわち Haskell Language Server も重要である。

HLS は、エディタや IDE に対して、

- 型情報の表示
- エラー表示
- 補完
- 定義ジャンプ
- import 補助
- リファクタリング支援

などを提供する。

VS Code、Neovim、Emacs などでは Haskell を書く場合、HLS が正しく動くかどうかで開発体験は大きく変わる。ただし、HLS はコンパイラそのものではない。実際の型検査やコンパイルには GHC が関わる。

付録 G 道具の階層関係

ここまでの道具を階層的に整理すると、次のようになる。

層	道具	役割
エディタ支援	HLS	型情報・補完・診断をエディタに提供する
プロジェクトビルド	Cabal / Stack	プロジェクト全体をビルド・実行・テストする
パッケージ記述	Cabal	ライブラリ・実行ファイル・依存関係を記述する
コンパイラ	GHC	Haskell コードをコンパイルする
対話環境	GHCi	小さな式や関数を対話的に試す
ツール管理	GHCup	GHC, Cabal, HLS, Stackなどを導入・切替する

この関係を簡単に図式化すると、次のようになる。

```

Editor / IDE
  |
  v
HLS
  |
  v
Cabal or Stack
  |
  v
GHC / GHCi
  ^
  |
GHCup installs and manages these tools

```

付録 H 本ノートのプロジェクトでの推奨方針

本ノートで扱っているような、ゲーム理論コードを研究資産として育てるプロジェクトでは、最初は次の方針が分かりやすい。

1. GHCup で GHC, Cabal, HLS を導入する。
2. Cabal プロジェクトとして `my-game-theory.cabal` を管理する。
3. `src/` にライブラリ本体を置く。
4. `app/` に薄い `Main.hs` を置く。
5. `test/` に理論的性質を確認するテストを書く。
6. 必要になった段階で Stack 対応を追加する。

つまり、最初からすべてのツールを使い分けようとする必要はない。まずは、

```

ghcup
ghc
cabal
hls

```

を基本セットとして理解すればよい。

その上で、複数人開発や環境固定が重要になった場合に Stack を導入する、という順序が自然である。

付録 I Appendix のまとめ

要点をまとめる。

- GHC は Haskell を実際にコンパイルする処理系である。
- GHCi は Haskell を対話的に試すための環境である。

- Cabal は Haskell プロジェクトの構造と依存関係を記述し、ビルドするための標準的な仕組みである。
- Stack は再現性を重視したプロジェクトビルドツールである。
- GHCup は GHC, Cabal, HLS, Stack などの導入とバージョン切替を担当する。
- HLS はエディタ連携を担う補助ツールであり、コンパイラそのものではない。

したがって、Haskell の開発環境は一つの巨大な道具ではなく、複数の層に分かれた道具群として理解するとよい。研究コードを育てる場合には、GHC が計算の中核を担い、Cabal がプロジェクト構造を記述し、GHCup が道具の導入と切替を支える、という関係を押さえておくことが重要である。

本ノートの主題であるモジュール分割や `src/app` 構成は、主として Cabal が管理するプロジェクト構造の上に成り立っている。したがって、Haskell の研究コードを資産化するには、単に GHC でコンパイルできることだけでなく、Cabal ファイルによってライブラリ、実行入口、テスト、依存関係を明示することが重要である。